

امتحان پایان ترم برنامه نویسی پیشرفته
دانشگاه خواجه نصیرالدین طوسی
دانشکده ریاضی و علوم کامپیوتر
۱۲۰ دقیقه

نام و نام خانوادگی.....
شماره دانشجویی.....

[برنامه نویسی شی گرا] قطعه کد زیر را در نظر بگیرید:

```
class MenuItem():
    """An instance represents an item on a menu."""
    def __init__(self, name, is_veggie, price):
        """A new menu item called name with 3 attributes:
        name: a non-empty str, e.g. 'Chicken Noodle Soup'
        is_veggie: a Bool indicating vegetarian or not
        price: an int > 0 """
        self.name = name
        self.is_veggie = is_veggie
        assert price > 0
        self.price = price

class LunchItem(MenuItem):
    """An instance represents an item that can also be served at lunch"""
    def __init__(self, name, is_veggie, price, lunch_price):
        """A menu item with one additional attribute:
        lunch_price: an int > 0 and <= 10"""
        super().__init__(name, is_veggie, price)
        assert lunch_price > 0
        assert lunch_price <= 10
        self.lunch_price = lunch_price
```

الف) یک دستور انتساب (assignment) در پایتون بنویسید که یک شیء جدید از کلاس MenuItem را در متغیر item1 ذخیره کند. نام این آیتم "Tofu Curry" است، یک غذای گیاهی که قیمت آن ۲۴ دلار است.

```
item1 = MenuItem("Tofu Curry", True, 24)
```

ب) یک دستور انتساب در پایتون بنویسید که یک شیء جدید از کلاس LunchItem را در متغیر item2 ذخیره کند. نام این آیتم "Hamburger" است، یک غذای غیرگیاهی که قیمت معمول آن ۱۲ دلار و قیمت ناهار آن ۸ دلار است.

```
item2 = LunchItem("Hamburger", False, 12, 8)
```

ج) قیمت ناهار نباید هرگز بیشتر از ۱۰ دلار باشد. متد __init__ این موضوع را تضمین می‌کند. یک خط کد پایتون بنویسید که نشان دهد چگونه این نامعادله هنوز می‌تواند شکسته شود.

```
item2.lunch price = 16
```

د) بدنه تابع audit_menu را مطابق با مشخصات زیر بنویسید.

```
def audit_menu (the_menu):
```

امتحان پایان ترم برنامه نویسی پیشرفته	نام و نام خانوادگی.....
دانشگاه خواجه نصیرالدین طوسی	شماره دانشجویی.....
دانشکده ریاضی و علوم کامپیوتر	
۱۲۰ دقیقه	

""Performs an audit of each LunchItem on the _menu, making sure that each lunch_price is never more than 10 dollars. A lunch_price of 11 dollars is changed to 9. An item whose lunch_price is more than 11 is too expensive to be offered at lunch; it must be replaced with a new, equivalent MenuItem (that has no lunch price). Items that are not LunchItems are unchanged. Modifies the _menu; does not create or return a new menu/list the _menu: possibly empty list of MenuItem ""

```
for i in list(range(len(the_menu))):
    item = the_menu[i]
    if isinstance(item, LunchItem):
        if item.lunch_price == 11:
            item.lunch_price = 9
        elif item.lunch_price > 11:
            newItem = MenuItem(item.name, item.is_veggie, item.price)
            the_menu[i] = newItem
```

[Regex]

۱. برای شناسایی پترن‌های معتبر با استفاده از regex، رنگ‌های هگزادسیمال ۲۴ و ۳۲ بیتی که با # شروع می‌شوند را برجسته کنید.

`#([0-9a-fA-F]{6})|([0-9a-fA-F]{8})`

مثال‌های نامعتبر:	مثال‌های معتبر:
<code>#00ffhh #agaeffe0</code>	<code>#ffffff, #ffffff, #00ff0088</code>

۲. در اعداد طولانی، کاماها را فقط در بخش عدد صحیح (قبل از اعشار) قرار دهید و بخش اعشاری را کاملاً نادیده بگیرید.

Input: 1234567890.12345 → 1,234,567,890.12345

`(\d)(?=(\d{3})+(\.\d*))`

[Numpy]

تابعی به نام compress بنویسید که یک تصویر خاکستری (آرایه‌ی NumPy با ابعاد (n, m)) را دریافت کند و هر بخش ۲×۲ از تصویر را با گرفتن میانگین مقادیر پیکسل‌ها فشرده‌سازی کند.

امتحان پایان ترم برنامه نویسی پیشرفته
دانشگاه خواجه نصیرالدین طوسی
دانشکده ریاضی و علوم کامپیوتر
۱۲۰ دقیقه

نام و نام خانوادگی.....
شماره دانشجویی.....

1	2	3	1
2	1	2	2
2	2	1	1
2	2	1	7



1.5	2
2	2.5

نیازی نیست نگران تبدیل مقادیر به اعداد صحیح باشید. می توانید فرض کنید تصویر داده شده مربع است و عرض و ارتفاع آن عددی زوج است. روش شما نباید آرایه‌ی ورودی را تغییر دهد، بلکه باید یک آرایه‌ی جدید برگرداند. برای گرفتن امتیاز کامل، راه حل شما باید فقط شامل دو حلقه باشد.

```
import numpy as np
def compress(image):
    n, m = image.shape
    new_n, new_m = n // 2, m // 2
    compressed = np.zeros((new_n, new_m), dtype=float)
    for i in range(new_n):
        for j in range(new_m):
            block = image[2*i:2*i+2, 2*j:2*j+2]
            compressed[i, j] = np.mean(block)
    return compressed
```

[Pandas]

یک DataFrame به نام dfrobot شامل مشخصات بیش از ۵۰ ربات در یک آزمایشگاه تحقیقاتی است. این DataFrame یک ستون شاخص (index) به نام Name دارد و ستون‌هایی برای رنگ، نوع حرکت و ارتفاع (بر حسب اینچ) هر ربات.

	Color	Motion	Hgt
Name			
RoboBee	Silver	Fly	1
R2-D2	White	Roll	43
C-3PO	Gold	Walk	69
...			

امتحان پایان ترم برنامه نویسی پیشرفته دانشگاه خواجه نصیرالدین طوسی دانشکده ریاضی و علوم کامپیوتر ۱۲۰ دقیقه	نام و نام خانوادگی..... شماره دانشجویی.....
---	--

کدی بنویسید که کارهای زیر را انجام دهد:

الف) تعداد ربات‌هایی که در `dfrobot` ارتفاعشان برابر با ۱ اینچ است را تعیین کنید.

`len(dfrobot[(dfrobot["Hgt"] == 1)])`

ب) نام تمام ربات‌های سفید رنگ و دارای حرکت غلتشی (`rolling`) را انتخاب کنید. نتیجه باید یک اندیس `pandas` یا لیستی از رشته‌ها باشد.

`dfrobot[(dfrobot["Motion"] == 'Roll') & (dfrobot["Color"] == 'White')].index`

ج) در میان ربات‌های راهرو (`walking`)، کوتاه‌ترین ربات را پیدا کنید و نام آن ربات را به صورت رشته‌ای مشخص کنید.

`dfrobot[(dfrobot["Motion"] == 'Walk')]["Hgt"].idxmin()`

You can also use `sortby Hgt`.

د) یک ربات زرد رنگ، با ارتفاع ۴۰ اینچ و حرکت غلتشی به نام `WALL-E` تازه وارد شده است. اطلاعات آن را به `dfrobot` اضافه کنید.

`dfrobot.loc['WALL-E'] = ['Yellow', 'Roll', 40]`

`append` or `concat` are also okay. There are multiple correct answers.

[Decorators] یک دکوراتور به نام `retry` بنویسید که وقتی تابع دکوراتور شده هنگام اجرا استثنا (`Exception`) پرتاب کرد، آن را مجدداً تا حداکثر `N` بار اجرا کند و اگر بعد از `N` بار هنوز خطا داشت، همان استثنا را بیرون بدهد.

شرایط:

- دکوراتور باید پارامتر `max_retries` (تعداد دفعات تلاش مجدد) را بپذیرد.
- دکوراتور باید بدون در نظر گرفتن امضای تابع اصلی، آن را با آرگومان‌های دلخواه فراخوانی کند.
- دکوراتور باید نام و مستندات تابع اصلی را حفظ کند.
- در هر بار تلاش، اگر خطا بود، پیام زیر چاپ شود:

Retrying <function_name>: attempt <current_attempt> failed with <exception_message>

`import functools`

`def retry(max_retries):`

امتحان پایان ترم برنامه نویسی پیشرفته
دانشگاه خواجه نصیرالدین طوسی
دانشکده ریاضی و علوم کامپیوتر
۱۲۰ دقیقه

نام و نام خانوادگی.....
شماره دانشجویی.....

```
def decorator(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        attempt = 0
        while True:
            try:
                return func(*args, **kwargs)
            except Exception as e:
                attempt += 1
                if attempt > max_retries:
                    raise
                print(f'Retrying {func.__name__}: attempt {attempt} failed with {e}')
        return wrapper
    return decorator
```

[Multhi-threading]

می‌خواهیم سیستمی طراحی کنیم که داده‌هایی (اعداد تصادفی) را از یک منبع دریافت کند (توسط Producer) و آن‌ها را در پایگاه داده فرضی ذخیره کند (توسط Consumer)

شما باید یک کلاس به نام Pipeline طراحی کنید که با کمک مکانیزم‌های هم‌زمان‌سازی در Python، ارتباط امن بین دو thread (تولیدکننده و مصرف‌کننده) را برقرار کند.

```
import random

SENTINEL = object()

def producer(pipeline):
    """Pretend we're getting a message from the network."""
    for index in range(10):
        message = random.randint(1, 101)
        print(f'Producer got message: {message}')
        pipeline.set_message(message, "Producer")

    # Send a sentinel message to tell consumer we're done
    pipeline.set_message(SENTINEL, "Producer")
```

```
def consumer(pipeline):
    """Pretend we're saving a number in the database."""
    message = 0
    while message is not SENTINEL:
        message = pipeline.get_message("Consumer")
        if message is not SENTINEL:
            print(f'Consumer storing message: {message}')
```

امتحان پایان ترم برنامه نویسی پیشرفته
دانشگاه خواجه نصیرالدین طوسی
دانشکده ریاضی و علوم کامپیوتر
۱۲۰ دقیقه

نام و نام خانوادگی.....
شماره دانشجویی.....

class Pipeline:

"""

Class to allow a single element pipeline between producer and consumer.

"""

def __init__(self):

self.message = 0

#TODO

def get_message(self, name):

#TODO

return message

def set_message(self, message, name):

#TODO

class Pipeline:

def __init__(self):

self.message = 0

self.producer_lock = threading.Lock()

self.consumer_lock = threading.Lock()

At start, consumer_lock is locked

so consumer must wait until producer sends first message

self.consumer_lock.acquire()

def set_message(self, message, who):

Producer must wait until consumer_lock is released

self.producer_lock.acquire()

print(f"{who} setting message: {message}")

self.message = message

Let consumer proceed

self.consumer_lock.release()

def get_message(self, who):

Consumer must wait until producer_lock is released

self.consumer_lock.acquire()

message = self.message

print(f"{who} got message: {message}")

Let producer proceed

نام و نام خانوادگی.....	امتحان پایان ترم برنامه نویسی پیشرفته
شماره دانشجویی.....	دانشگاه خواجه نصیرالدین طوسی
	دانشکده ریاضی و علوم کامپیوتر
	۱۲۰ دقیقه

```
self.producer_lock.release()  
return message
```

موفق باشید!! :